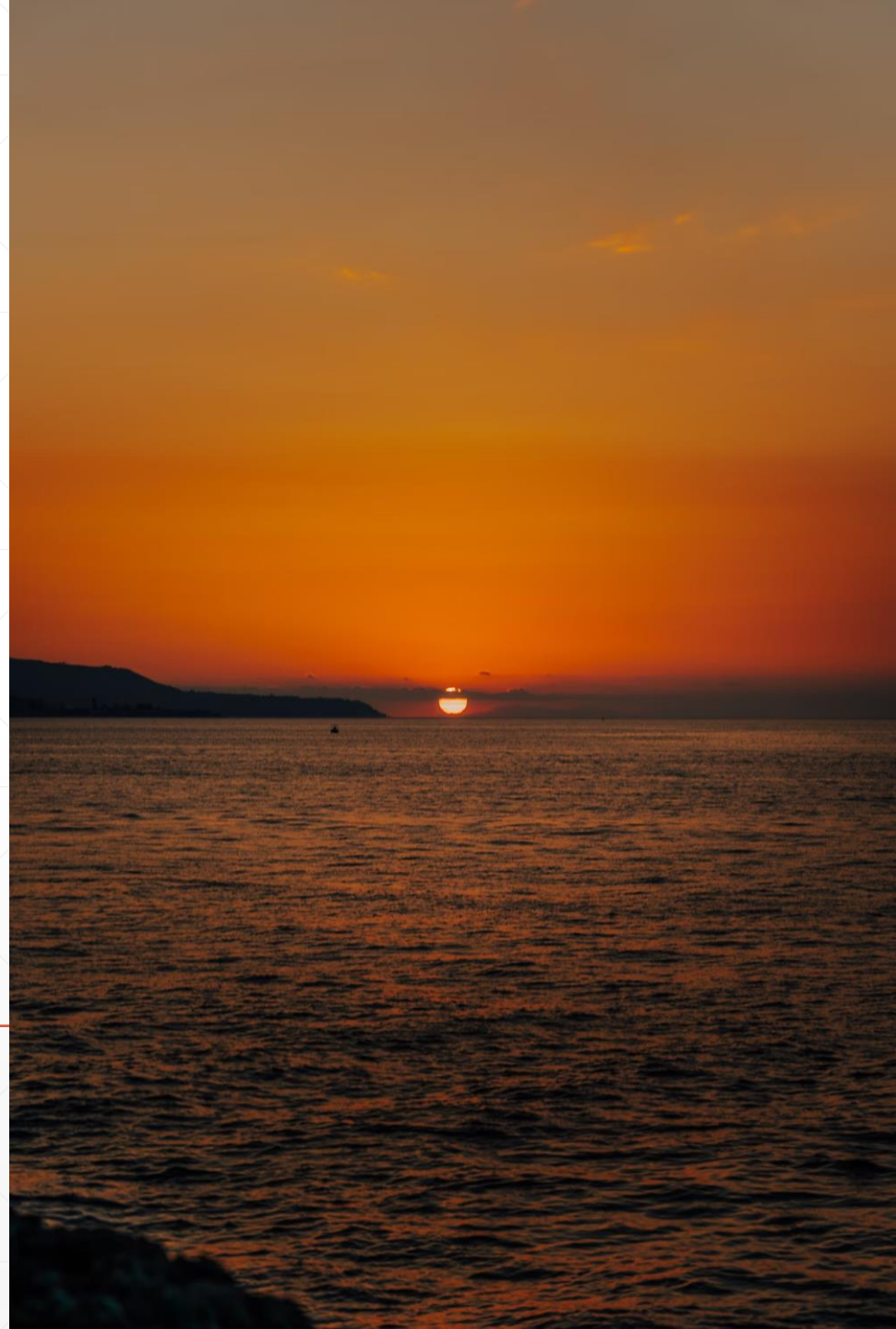


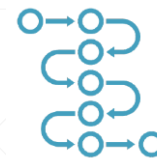
使用Netty开发高性能网络应用

预备知识

北京理工大学计算机学院
金旭亮



方法的“同步”与“异步”调用



```
void process(){  
    ...  
    f();  
    g();  
}
```

方法的同步调用，是指代码在执行一个方法调用时，必须等其执行完毕，才能执行下一句，如左图所示，f()不执行完，g()不可能执行，整个线程会“阻塞”在这里等待f()执行结束。

```
void async processAsync(){  
    ...  
    await f();  
    await g();  
}
```

方法的异步调用，是指代码在执行一个方法调用时，不必等f()执行完毕，而是在这里“作个记号”后，调用者线程立即返回，不会阻塞等待f()的执行结束，这个，通常又称为“挂起”。

f()的执行结束之后，系统会“回到”原先打了记号的地方，开始执行g()，而g()的执行方式，也与f()一样，不会阻塞当前线程，这是当前许多异步编程框架所采用编程模式。

在实际开发中，与异步调用紧密关联的，是“事件驱动”这种运行模式。“事件驱动”的软件系统，通常会有一个特殊的“消息循环”线程，负责触发各种事件，开发者事先针对感兴趣的事件写好事件响应代码，然后，当此事件触发时，系统会自动地“回调”这些写好的事件响应代码。Netty的异步编程框架，其实就混杂了“事件驱动”与“异步调用”这两种模式。

“阻塞 / 非阻塞”操作



阻塞与非阻塞，主要是针对I/O操作来说的。

当应用程序启动一个I/O操作之后，如果执行线程必须“停下来”等待I/O操作的完成，才能执行后面的代码，我们就说，这是一个“阻塞”式的I/O操作。

如果执行线程不需要“停下来”等待I/O操作的完成，而只是做了相应的处理（比如注册一个回调函数，等I/O操作完成之后再调用）之后，就立即返回而不会阻塞，我们就说，这是一个“非阻塞”式的I/O操作。



现代操作系统，通常同时提供“阻塞”和“非阻塞”两套系统调用API。

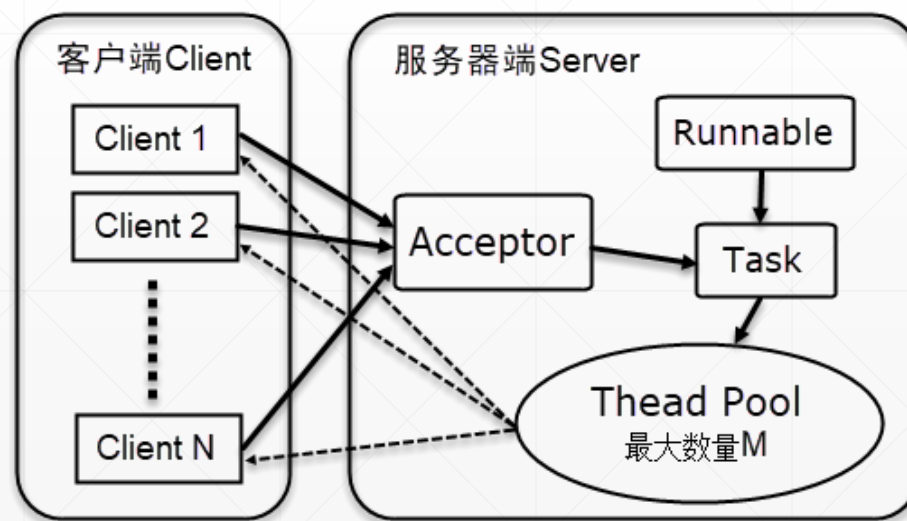
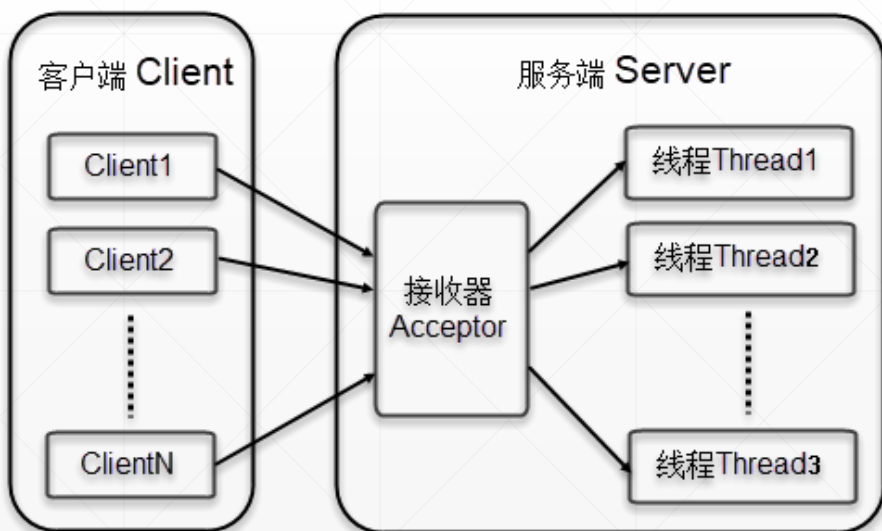
JDK中的网络组件



最早的Java网络API（放在java.net包中），是由本地操作系统套接字（Socket）所支持的，采用的是同步阻塞式编程模型。服务端需要new一个ServerSocket，然后阻塞调用accept()方法等待连接，客户端需要new一个Socket，调用connect()方法连接上Server，这样，数据通道才能建立，之后，就可以调用read和write系列方法来读写数据。



采用这种开发方式，需要通过多线程来提供并发能力，每个线程负责一个客户端，为了降低系统资源占用，通常会引入线程池。



Java NIO-1

现代操作系统的本地套接字库很早就提供了非阻塞调用，Java对于非阻塞I/O的支持是在2002年引入的，位于JDK 1.4的java.nio包中。



NIO 最开始是新的输入/输出 (New Input/Output) 的英文缩写，但是，该Java API 已经出现足够长的时间了，不再是“新的”了，因此，如今大多数的用户认为NIO 代表非阻塞 I/O (Non-blocking I/O)，而阻塞I/O (blocking I/O) 是旧的输入/输出 (old input/output, OIO)。

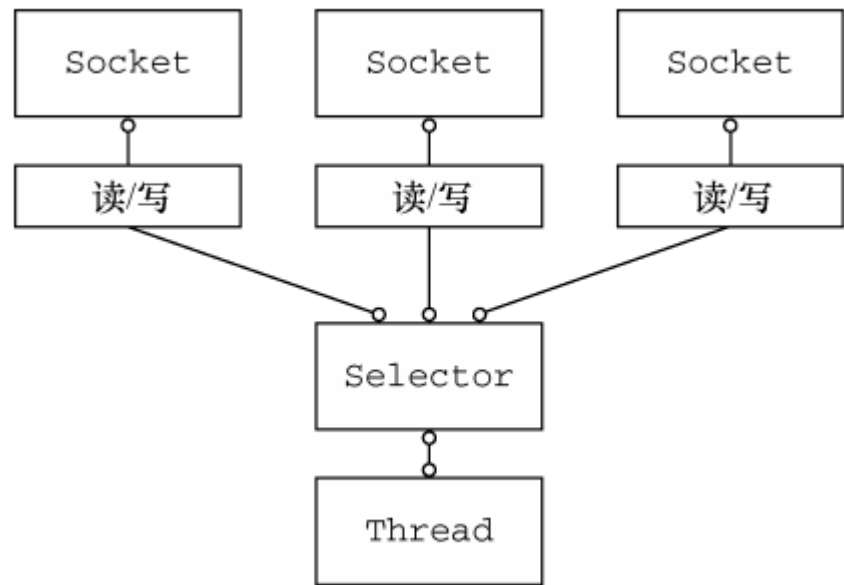
Java NIO-2

Java NIO中的核心组件是Selector，它采用的是基于“循环检测”+“回调”实现的编程模式。使用一个单一的线程，便可以处理多个并发的连接。

NIO将数据传送过程抽象为一个异步非阻塞的“通道（Channel）”，并为之提供了一个数据容器——“字节缓冲区（ByteBuffer）”。

使用NIO可以开发出高性能的网络应用，但问题在于它的API相当复杂，学习曲线陡峭，写出来的代码，如果事先没有精心设计，难以测试、调试和维护，而且没有为开发中常见的应用场景提供“开箱即用”的网络组件，一切都得“自己来”，开发工作量大。

Netty本质上是对NIO组件的封装，设计了一套新的编程组件，弥补了NIO的上述缺陷，同时，又将其“高性能”的优势发挥得淋漓尽致。



Java AIO



Java AIO (Java Asynchronous IO) 模式是在Java 1.7版本中对NIO模式的一种改进，因此也被称为NIO 2。



AIO的主要特性是引入了“事件驱动”的异步非阻塞式编程模型，但它本身还是比较复杂，其编程模型与NIO有较大的差异（可以看成是两种编程模型），性能上并没有多大的改善。



在实际开发中，由于已经有了Netty这种被广泛使用的网络框架，所以AIO就没能流行开来，对于这一技术，有所了解就行了，通常没有必要精学。

Java生态圈中，Netty是赢家！



当前互联网应用的后端，普遍地应用了微服务架构，一些比较复杂的后端，包容有上百个微服务。



当系统变得复杂，微服务个数上升，微服务之间的相互调用也变得越发频繁，这就要求这些服务间的调用，应该是异步非阻塞的，否则，整个系统的稳定性及吞吐量，会受到很大的影响，遇到稍微高一些的并发量，系统就可能崩溃。



正是这种时代背景，对高性能的异步开发框架，提出了强劲的需求，出现了不少相应的开发框架，在这些框架“大战”中，Netty是非常引人注目的一个，在Java平台，基本上可以认为它就是赢家，非常多的系统使用了它，也有非常多的框架基于它进行二次开发。